Advanced machine learning and data analysis for the physical sciences

Morten Hjorth-Jensen[1]

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway[1]

March 5

© 1999-2026, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week of March 2-6

1. Reminder on basics of recurrent neural networks (RNNs)
2. Mathematics of RNNs
3. L(ong)S(hort)T(erm)M(emory)s and RNNs
4. Writing our own codes for RNNs
5. Lab session: discussion and work on projects
6. Video of lecture at https://youtu.be/6wYD12g5kfU
7. Whiteboard notes at https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/HandwrittenNotes/2026/Notesweek7.pdf
8. Reading recommendations:
   8.1 Goodfellow, Bengio and Courville's chapter 10 from Deep Learning
   8.2 Sebastian Rashcka et al, chapter 15, Machine learning with Sickit-Learn and PyTorch
   8.3 David Foster, Generative Deep Learning with TensorFlow, see chapter 5

What is a recurrent NN?

A recurrent neural network (RNN), as opposed to a regular fully connected neural network (FCNN) or just neural network (NN), has layers that are connected to themselves.
In an FCNN there are no connections between nodes in a single layer. For instance, $(h_1^1$ is not connected to $(h_2^1$. In addition, the input and output are always of a fixed length.
In an RNN, however, this is no longer the case. Nodes in the hidden layers are connected to themselves.

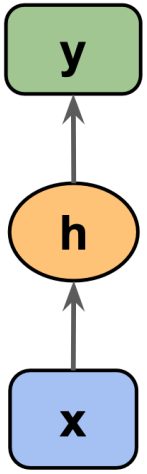
Why RNNs?

Recurrent neural networks work very well when working with sequential data, that is data where the order matters. In a regular fully connected network, the order of input doesn't really matter. Another property of RNNs is that they can handle variable input and output. Consider again the simplified breast cancer dataset. If you have trained a regular FCNN on the dataset with the two features, it makes no sense to suddenly add a third feature. The network would not know what to do with it, and would reject such inputs with three features (or any other number of features that isn't two, for that matter).

Basic layout, Figures from Sebastian Rashcka et al, Machine learning with Sickit-Learn and PyTorch

Overview

Networks we used previously: also called feedforward neural networks



Recurrent Neural Network (RNN)

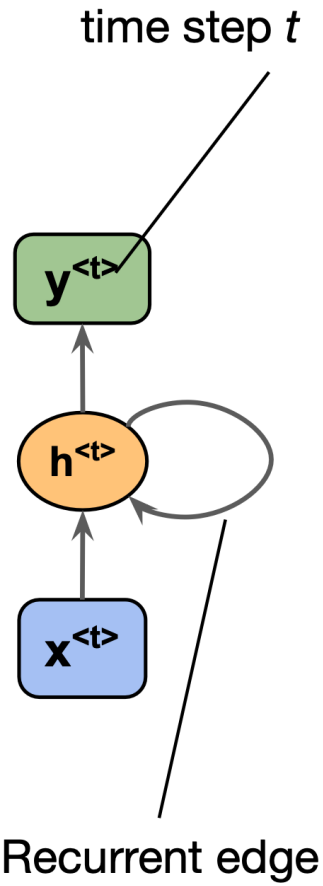


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

RNNs in more detail

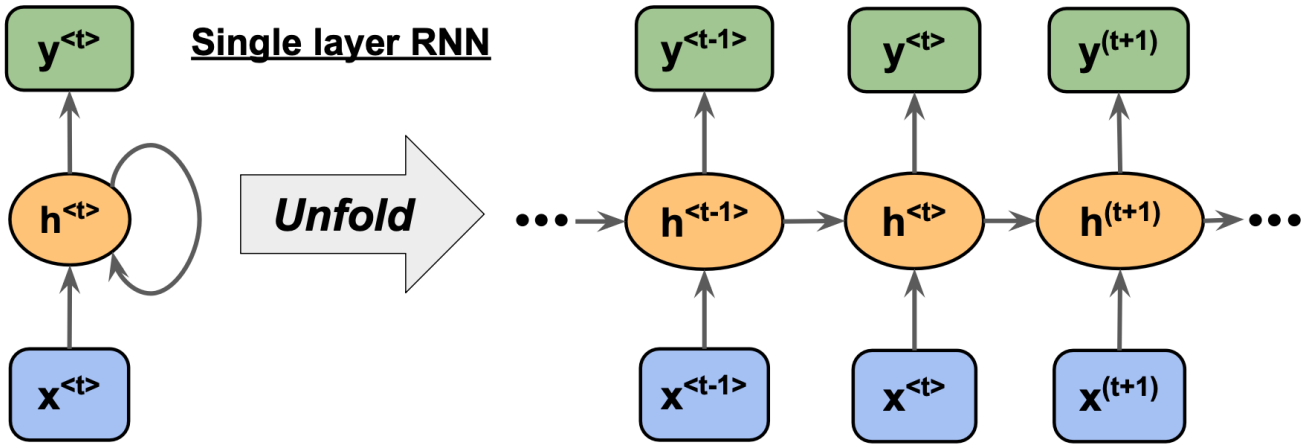
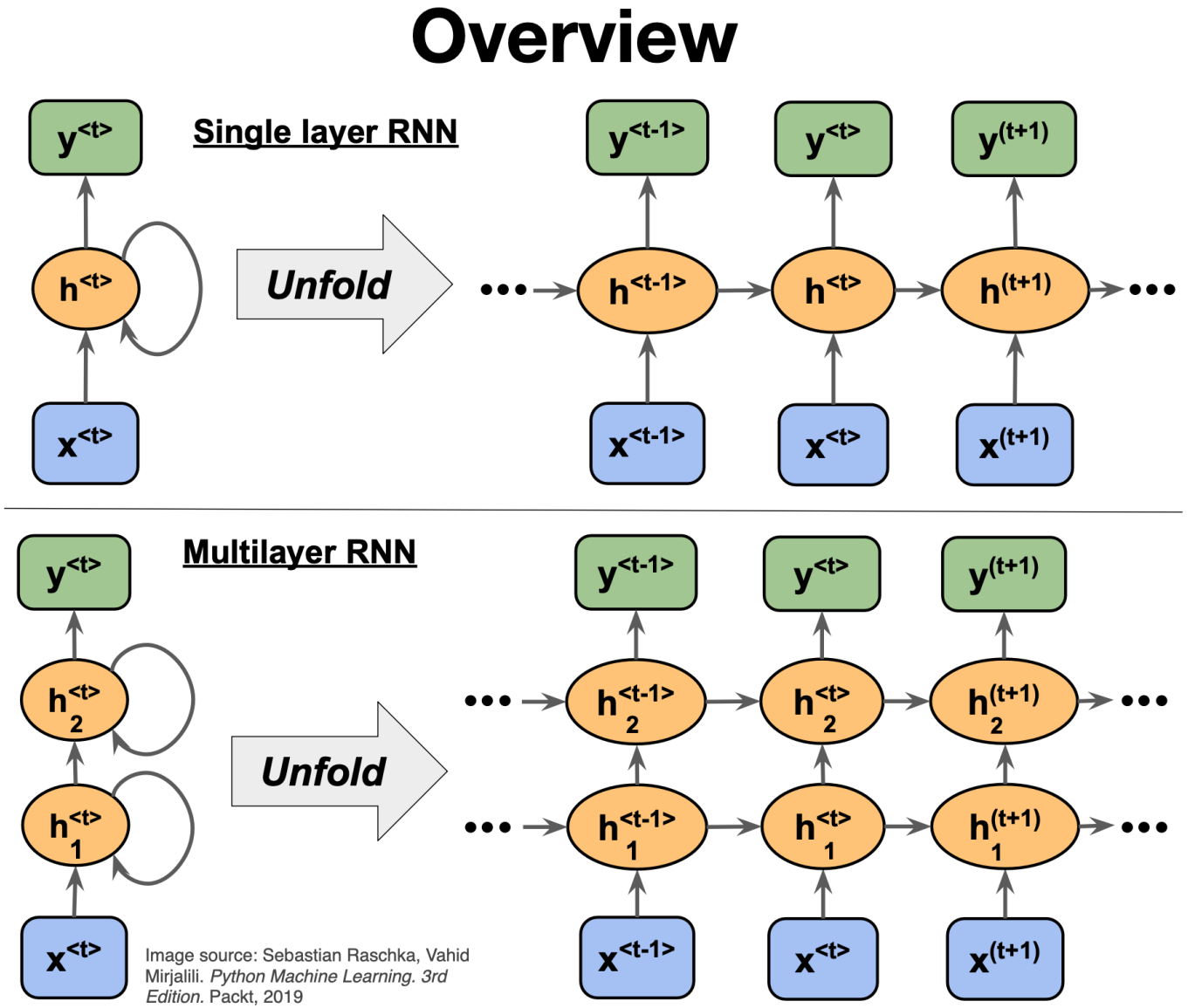


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

RNNs in more detail, part 2



RNNs in more detail, part 3

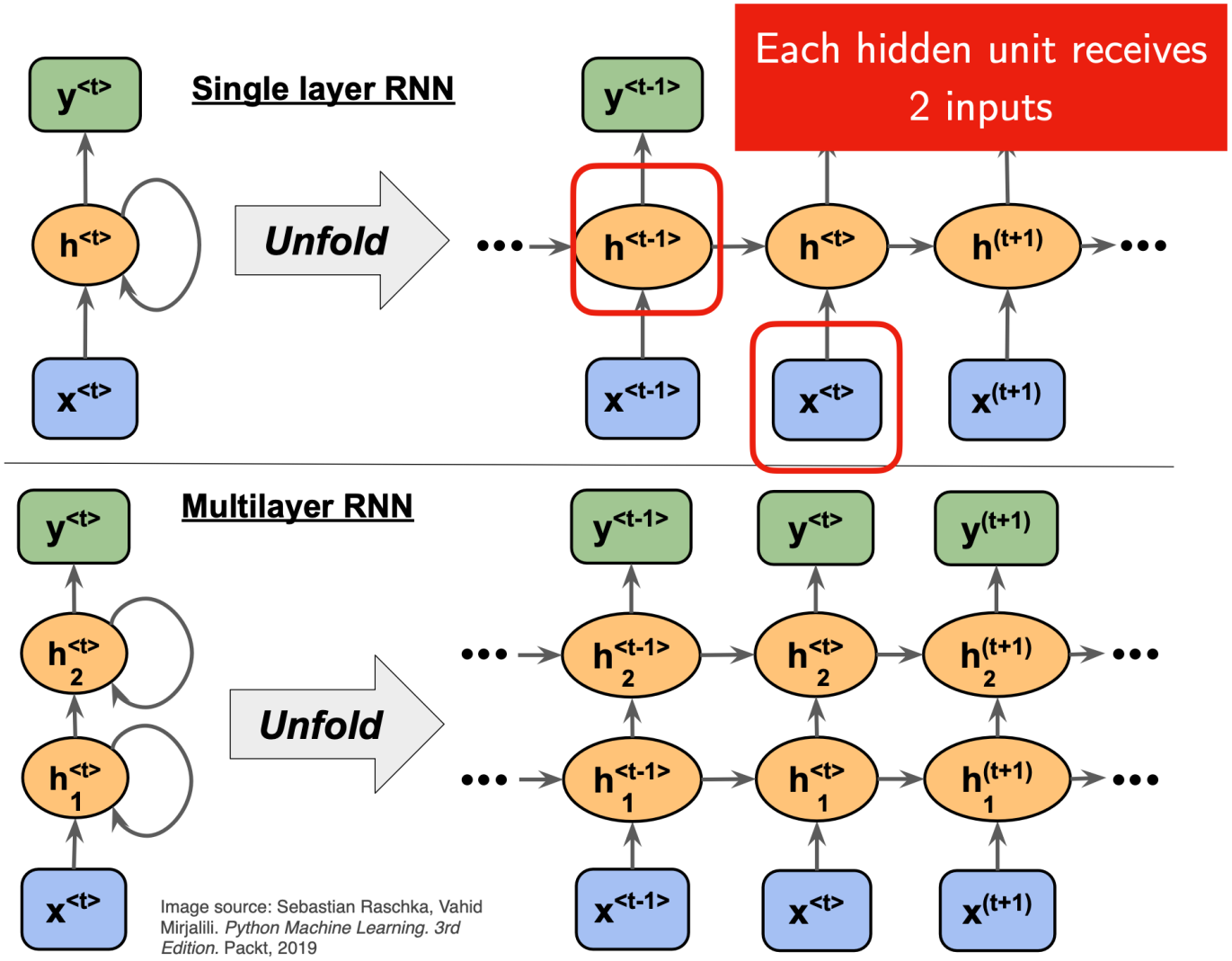
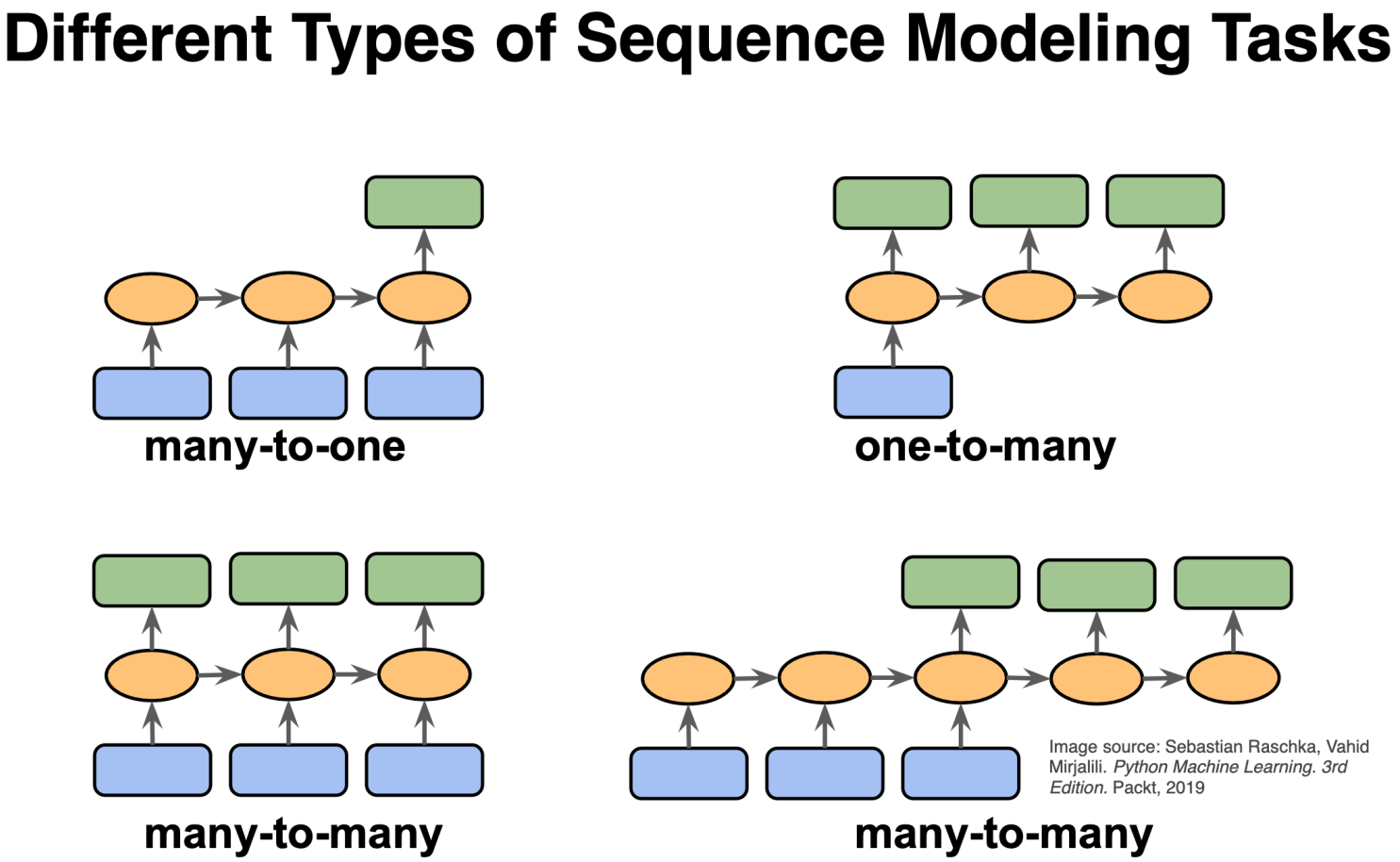


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

RNNs in more detail, part 4



RNNs in more detail, part 5

Weight matrices in a single-hidden layer RNN

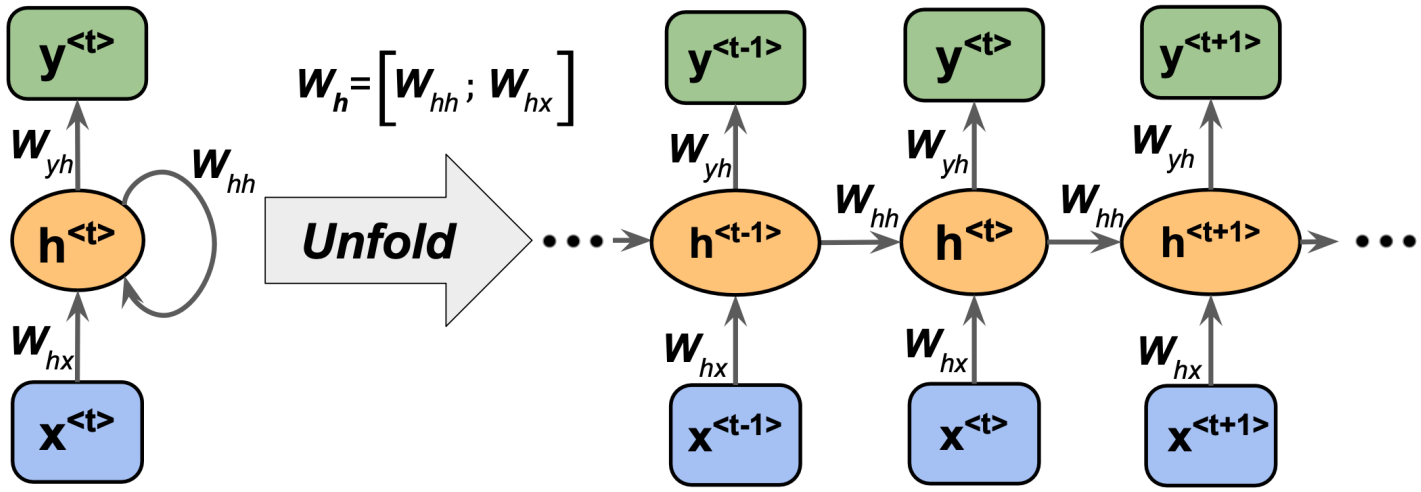


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

RNNs in more detail, part 6

Weight matrices in a single-hidden layer RNN

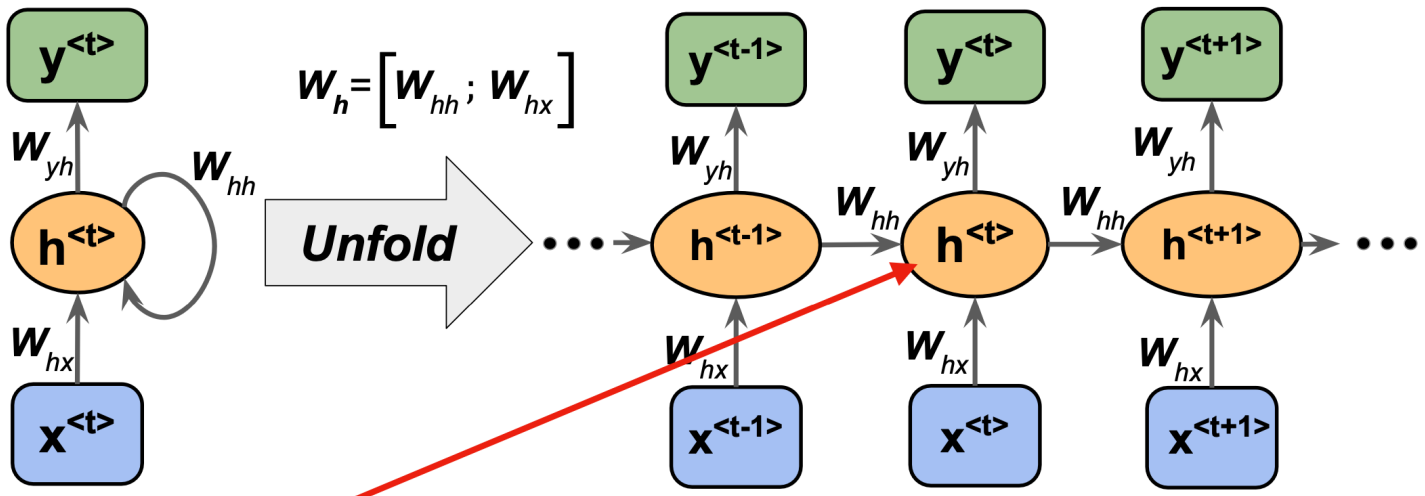


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

Net input:

$$\mathbf{z}_h^{\langle t \rangle} = \mathbf{W}_{hx}\mathbf{x}^{\langle t \rangle} + \mathbf{W}_{hh}\mathbf{h}^{\langle t-1 \rangle} + \mathbf{b}_h$$

Activation:

$$\mathbf{h}^{\langle t \rangle} = \sigma_h\left(\mathbf{z}_h^{\langle t \rangle}\right)$$

RNNs in more detail, part 7

Weight matrices in a single-hidden layer RNN

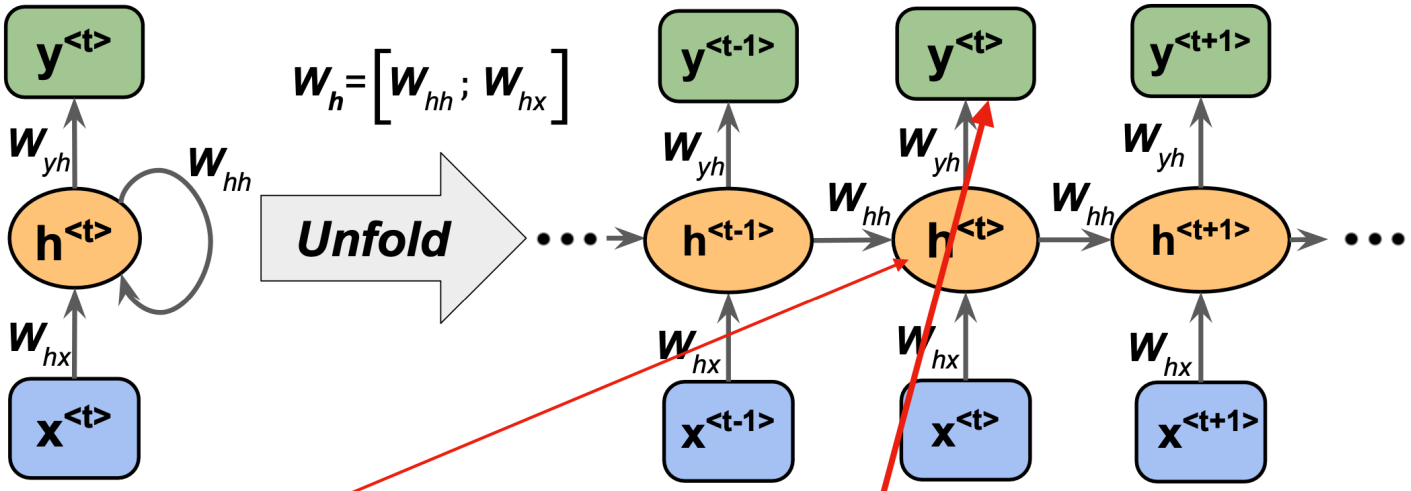


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

Net input:

$$\mathbf{z}_h^{\langle t \rangle} = \mathbf{W}_{hx}\mathbf{x}^{\langle t \rangle} + \mathbf{W}_{hh}\mathbf{h}^{\langle t-1 \rangle} + \mathbf{b}_h$$

Activation:

$$\mathbf{h}^{\langle t \rangle} = \sigma_h\left(\mathbf{z}_h^{\langle t \rangle}\right)$$

Net input:

$$\mathbf{z}_y^{\langle t \rangle} = \mathbf{W}_{yh}\mathbf{h}^{\langle t \rangle} + \mathbf{b}_y$$

Output:

$$\mathbf{y}^{\langle t \rangle} = \sigma_y\left(\mathbf{z}_y^{\langle t \rangle}\right)$$

Backpropagation through time

We can think of the recurrent net as a layered, feed-forward net with shared weights and then train the feed-forward net with weight constraints.

We can also think of this training algorithm in the time domain:

1. The forward pass builds up a stack of the activities of all the units at each time step.
2. The backward pass peels activities off the stack to compute the error derivatives at each time step.
3. After the backward pass we add together the derivatives at all the different times for each weight.

The backward pass is linear

1. There is a big difference between the forward and backward passes.
2. In the forward pass we use squashing functions (like the logistic) to prevent the activity vectors from exploding.
3. The backward pass, is completely linear. If you double the error derivatives at the final layer, all the error derivatives will double.

The forward pass determines the slope of the linear function used for backpropagating through each neuron

The problem of exploding or vanishing gradients

- What happens to the magnitude of the gradients as we backpropagate through many layers?
  1. If the weights are small, the gradients shrink exponentially.
  2. If the weights are big the gradients grow exponentially.
- Typical feed-forward neural nets can cope with these exponential effects because they only have a few hidden layers.
- In an RNN trained on long sequences (e.g. 100 time steps) the gradients can easily explode or vanish.
  1. We can avoid this by initializing the weights very carefully.
- Even with good initial weights, its very hard to detect that the current target output depends on an input from many time-steps ago.

RNNs have difficulty dealing with long-range dependencies.

The mathematics of RNNs, the basic architecture

See notebook at https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week7/ipynb/rnnmath.ipynb

Four effective ways to learn an RNN and preparing for next week

1. Long Short Term Memory Make the RNN out of little modules that are designed to remember values for a long time.
2. Hessian Free Optimization: Deal with the vanishing gradients problem by using a fancy optimizer that can detect directions with a tiny gradient but even smaller curvature.
3. Echo State Networks (ESN): Initialize the input a hidden and hidden-hidden and output-hidden connections very carefully so that the hidden state has a huge reservoir of weakly coupled oscillators which can be selectively driven by the input. ESNs only need to learn the hidden-output connections.
4. Good initialization with momentum Initialize like in Echo State Networks, but then learn all of the connections using momentum

Long Short Term Memory (LSTM)

LSTM uses a memory cell for modeling long-range dependencies and avoid vanishing gradient problems.

1. Introduced by Hochreiter and Schmidhuber (1997) who solved the problem of getting an RNN to remember things for a long time (like hundreds of time steps).
2. They designed a memory cell using logistic and linear units with multiplicative interactions.
3. Information gets into the cell whenever its "write" gate is on.
4. The information stays in the cell so long as its **keep** gate is on.
5. Information can be read from the cell by turning on its **read** gate.

Gating mechanism: Long Short Term Memory (LSTM)

Besides a simple recurrent neural network layer, as discussed during the last two weeks, there are two other commonly used types of recurrent neural network layers: Long Short Term Memory (LSTM) and Gated Recurrent Unit (GRU). For a short introduction to these layers see https://medium.com/mindboard/lstm-vs-gru-experimental-comparison-955820c21e8b and https://medium.com/mindboard/lstm-vs-gru-experimental-comparison-955820c21e8b.

LSTM uses a memory cell for modeling long-range dependencies and avoid vanishing gradient problems. Capable of modeling longer term dependencies by having memory cells and gates that controls the information flow along with the memory cells.

1. Introduced by Hochreiter and Schmidhuber (1997) who solved the problem of getting an RNN to remember things for a long time (like hundreds of time steps).
2. They designed a memory cell using logistic and linear units with multiplicative interactions.
3. Information gets into the cell whenever its "write" gate is on.
4. The information stays in the cell so long as its **keep** gate is on

Implementing a memory cell in a neural network

To preserve information for a long time in the activities of an RNN, we use a circuit that implements an analog memory cell.

1. A linear unit that has a self-link with a weight of 1 will maintain its state.
2. Information is stored in the cell by activating its write gate.
3. Information is retrieved by activating the read gate.
4. We can backpropagate through this circuit because logistics are have nice derivatives.

LSTM details

The LSTM is a unit cell that is made of three gates:

1. the input gate,
2. the forget gate,
3. and the output gate.

It also introduces a cell state c, which can be thought of as the long-term memory, and a hidden state h which can be thought of as the short-term memory.

Basic layout (All figures from Raschka *et al.*,)

Long-short term memory (LSTM)

LSTM cell:

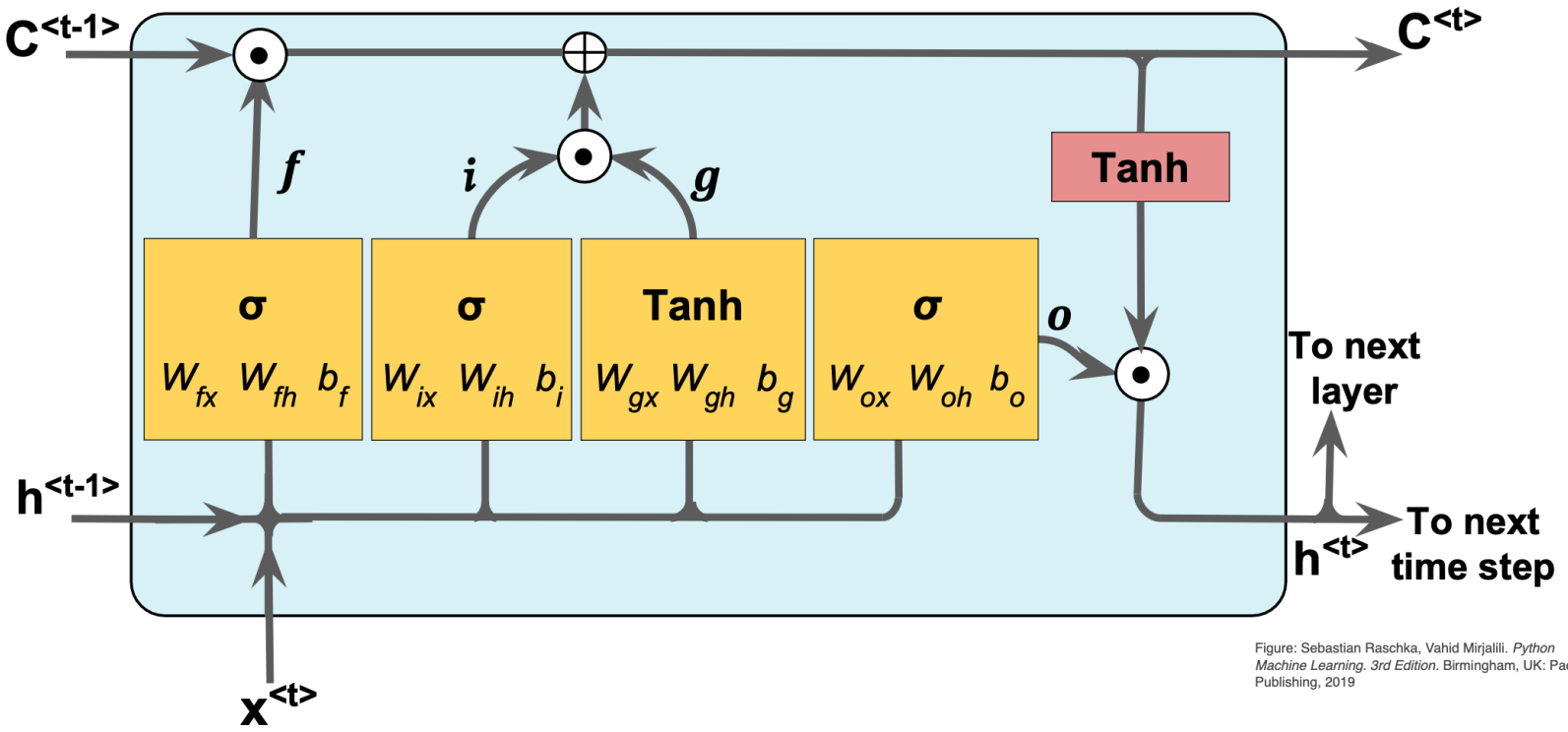


Figure: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Birmingham, UK: Packt Publishing, 2019

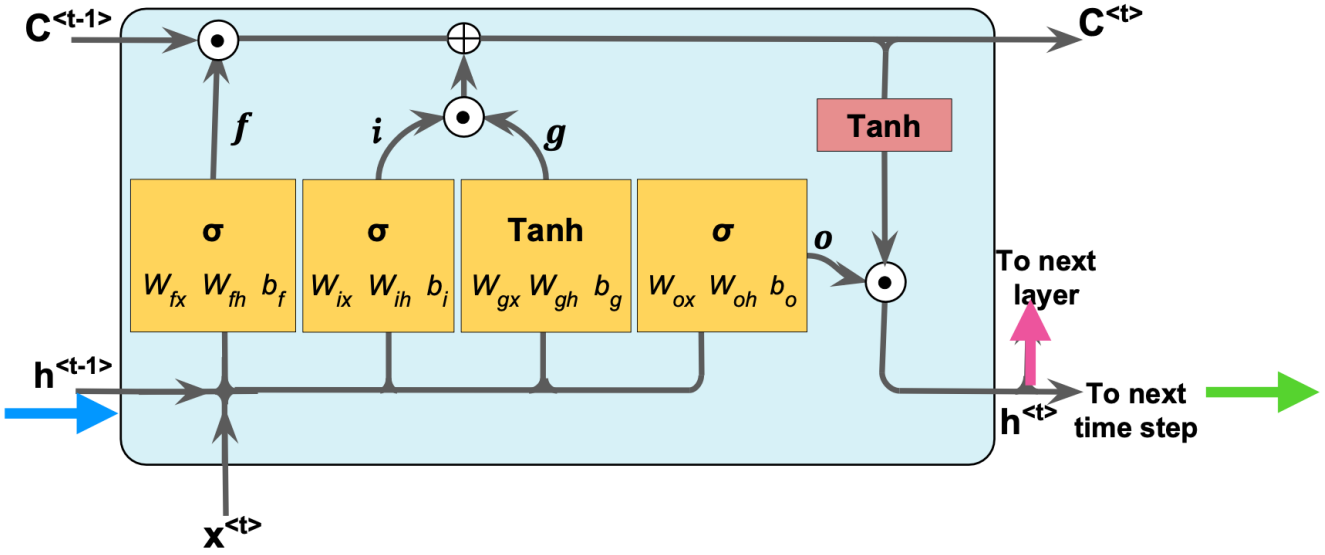
LSTM details

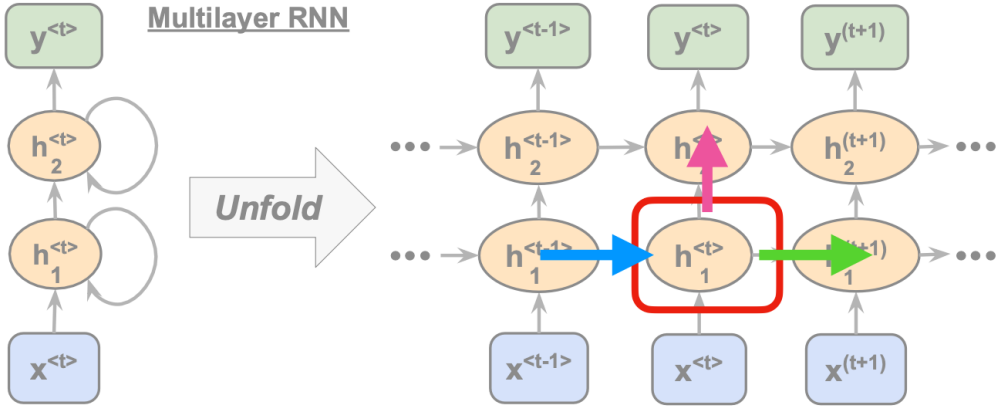
The first stage is called the forget gate, where we combine the input at (say, time t), and the hidden cell state input at $t-1$, passing it through the Sigmoid activation function and then performing an element-wise multiplication, denoted by $\odot$. Mathematically we have (see also figure below)

$$\mathrm{f}^{(t)} = \sigma(W_{fx}\mathrm{x}^{(t)} + W_{fh}\mathrm{h}^{(t-1)} + \mathrm{b}_f)$$

where the Ws are the weights to be trained.

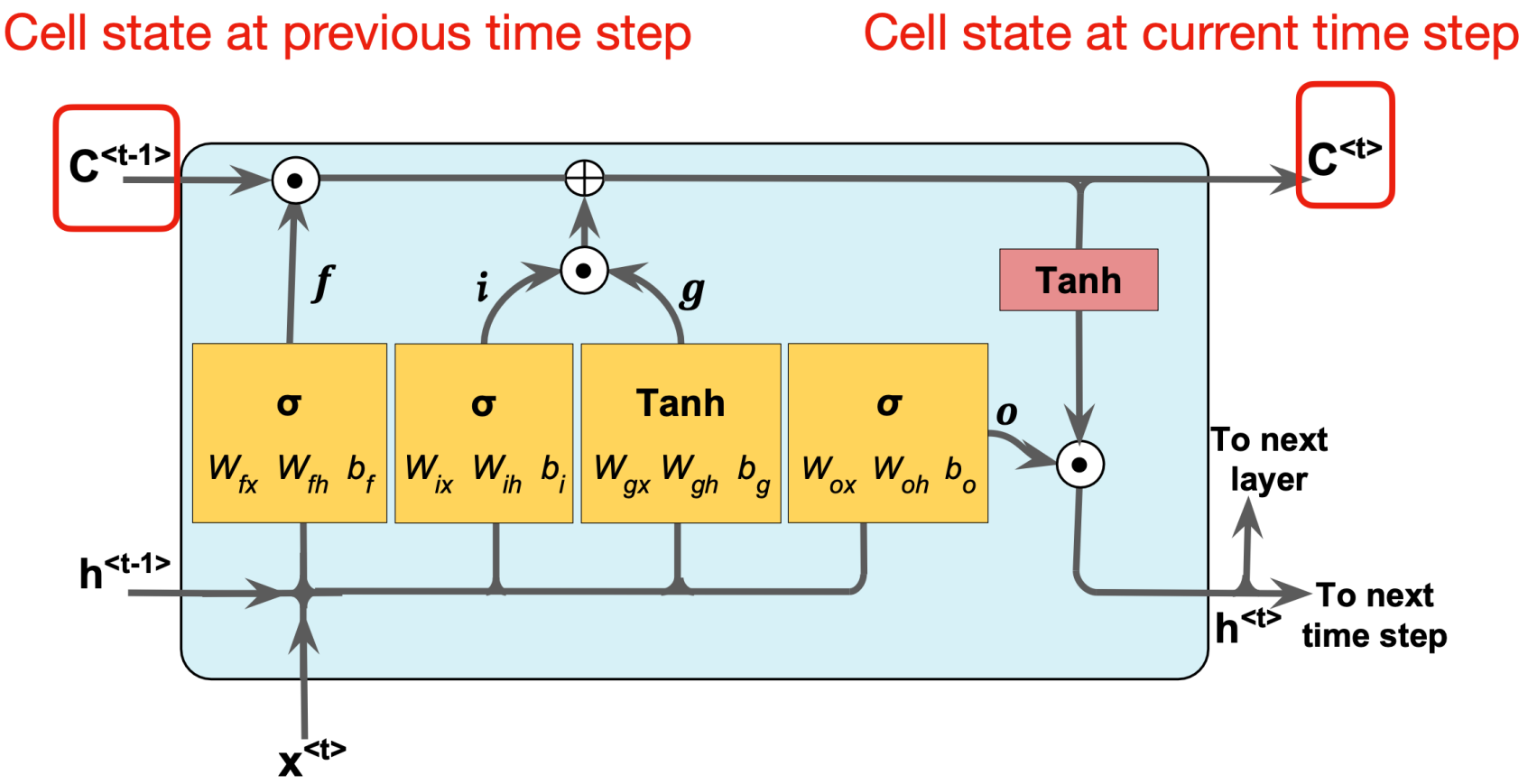
Comparing with a standard RNN



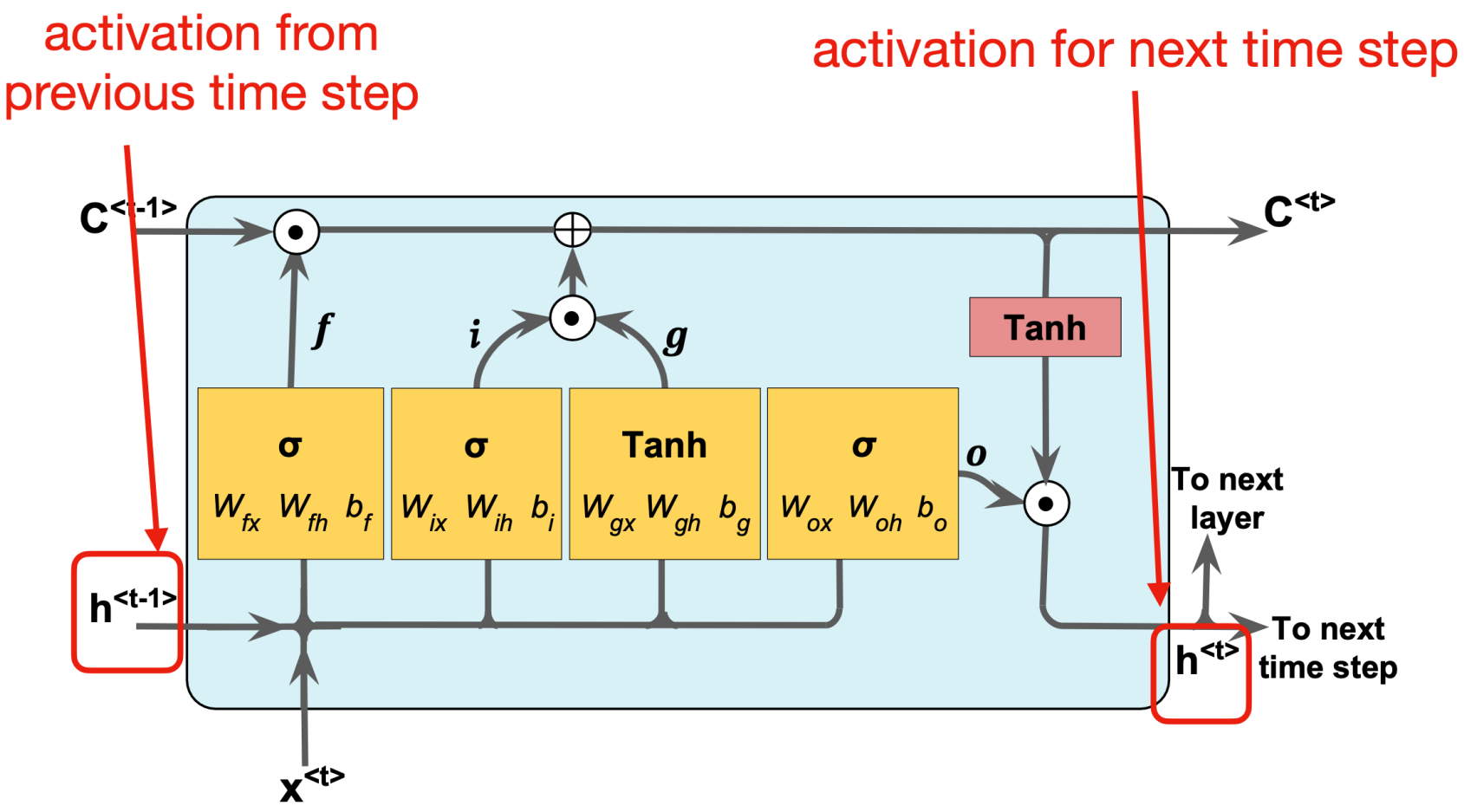


LSTM details I

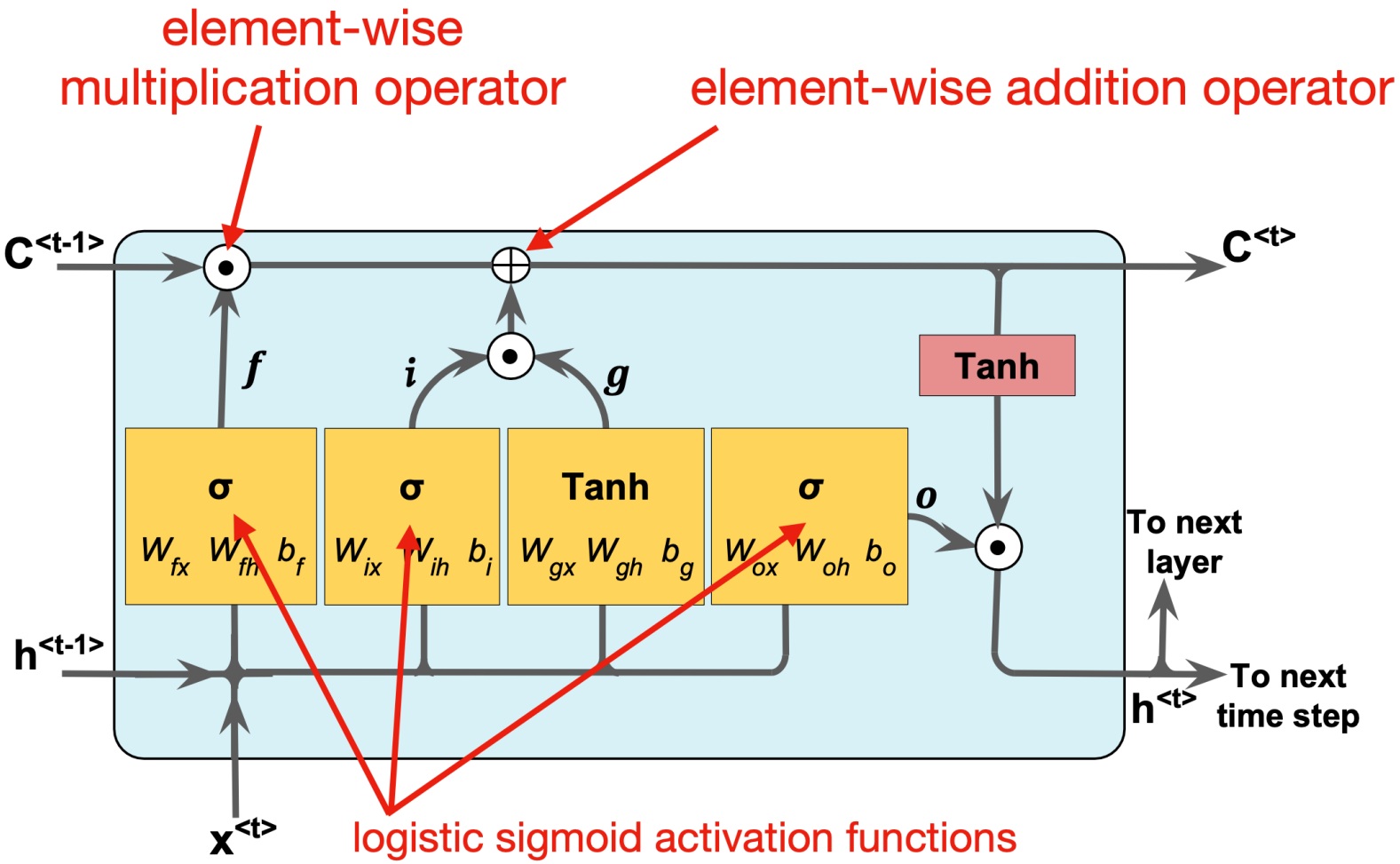
Long-short term memory (LSTM)



LSTM details II



LSTM details III



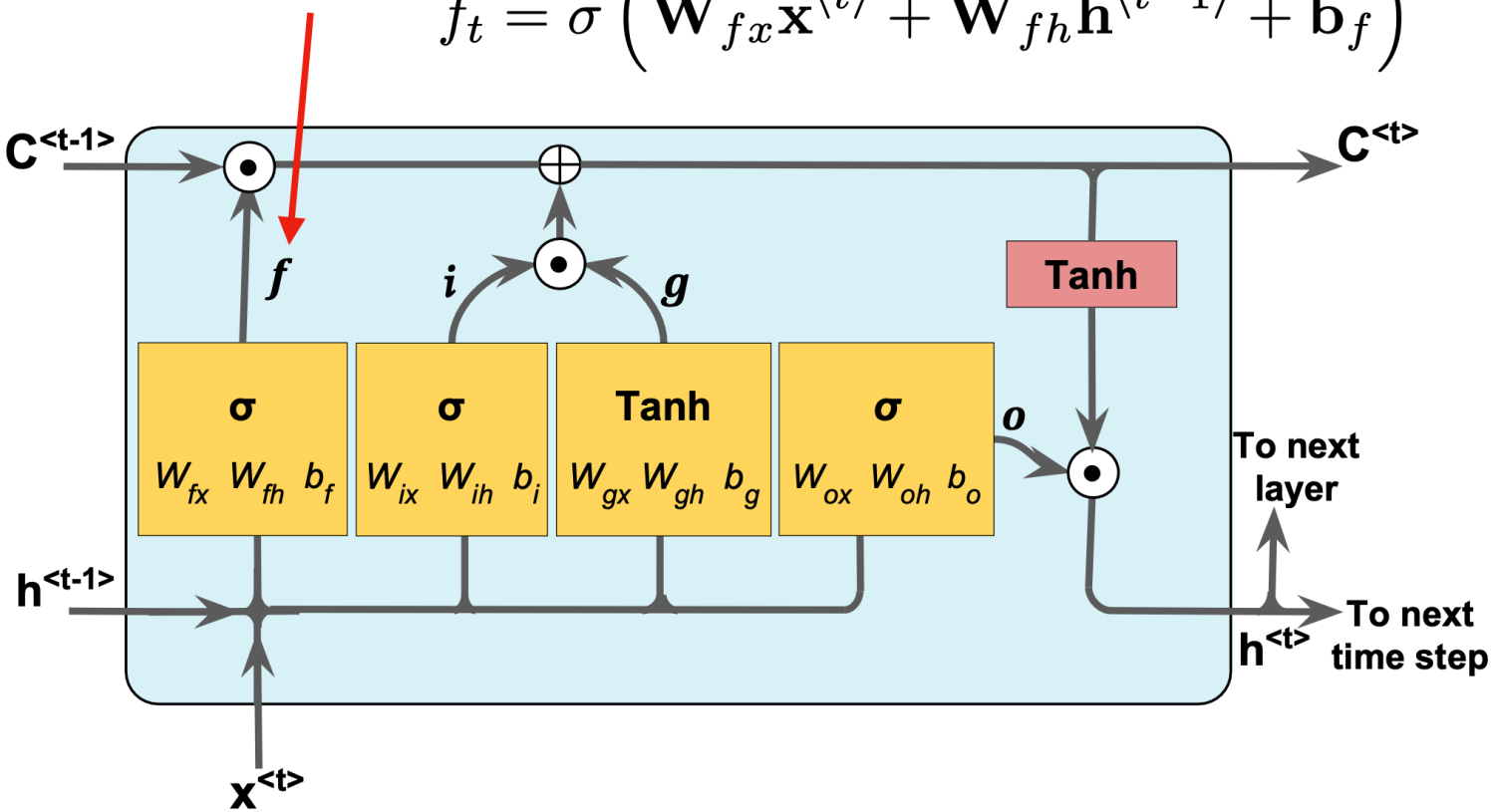
Forget gate

Long-short term memory (LSTM)

Gers, Felix A., Jürgen Schmidhuber, and Fred Cummins. "Learning to forget: Continual prediction with LSTM." (1999): 850-855.

"Forget Gate": controls which information is remembered, and which is forgotten; can reset the cell state

$$f_t = \sigma\left(\mathbf{W}_{fx}\mathbf{x}^{\langle t\rangle} + \mathbf{W}_{fh}\mathbf{h}^{\langle t-1\rangle} + \mathbf{b}_f\right)$$



The forget gate

The naming forget gate stems from the fact that the Sigmoid activation function's outputs are very close to 0 if the argument for the function is very negative, and 1 if the argument is very positive. Hence we can control the amount of information we want to take from the long-term memory.

$$\mathrm{f}^{(t)} = \sigma(W_{fx}\mathrm{x}^{(t)} + W_{fh}\mathrm{h}^{(t-1)} + \mathrm{b}_f)$$

where the Ws are the weights to be trained.

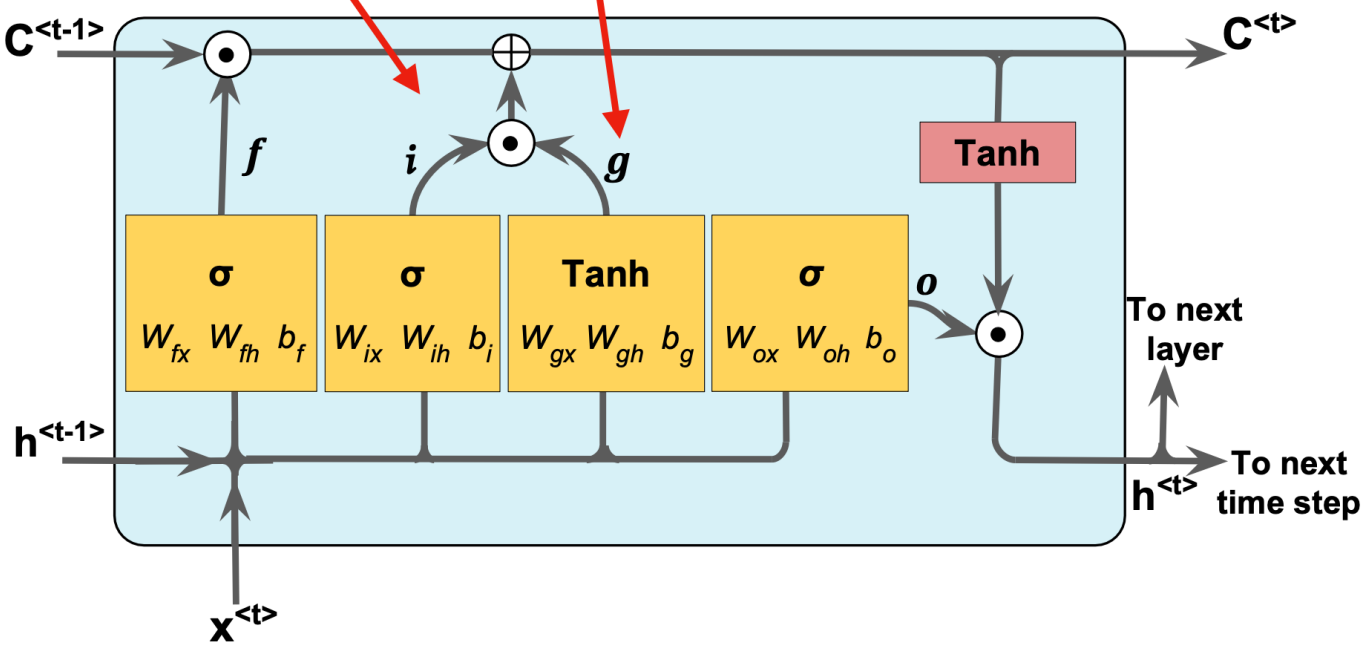
Basic layout

Long-short term memory (LSTM)

"Input Gate": $\mathbf{i}_t = \sigma\left(\mathbf{W}_{ix}\mathbf{x}^{\langle t\rangle} + \mathbf{W}_{ih}\mathbf{h}^{\langle t-1\rangle} + \mathbf{b}_i\right)$

"Input Node":

$$\mathbf{g}_t = \tanh\left(\mathbf{W}_{gx}\mathbf{x}^{\langle t\rangle} + \mathbf{W}_{gh}\mathbf{h}^{\langle t-1\rangle} + \mathbf{b}_g\right)$$



Input gate

The next stage is the input gate, which consists of both a Sigmoid function (σ_i), which decide what percentage of the input will be stored in the long-term memory, and the $\tanh_i$ function, which decide what is the full memory that can be stored in the long term memory. When these results are calculated and multiplied together, it is added to the cell state or stored in the long-term memory, denoted as $\oplus$.

We have

$$\mathrm{i}^{(t)} = \sigma_g(W_{ix}\mathrm{x}^{(t)} + W_{ih}\mathrm{h}^{(t-1)} + \mathrm{b}_i),$$

and

$$\mathrm{g}^{(t)} = \tanh(W_{gx}\mathrm{x}^{(t)} + W_{gh}\mathrm{h}^{(t-1)} + \mathrm{b}_g),$$

again the Ws are the weights to train.

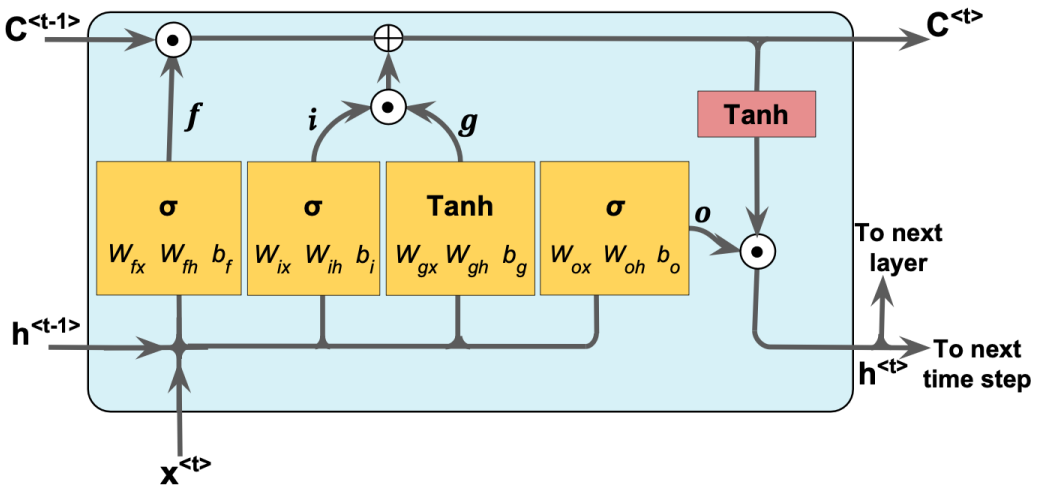
Short summary

Long-short term memory (LSTM)

Brief summary of the gates so far ...

Forget Gate Input Node Input Gate

$$C^{\langle t \rangle} = \left(C^{\langle t-1 \rangle} \odot f_t \right) \oplus \underbrace{\left(i_t \odot g_t \right)}_{\text{For updating the cell state}}$$



Forget and input

The forget gate and the input gate together also update the cell state with the following equation,

$$\mathsf{c}^{(t)} = \mathsf{f}^{(t)} \otimes \mathsf{c}^{(t-1)} + \mathsf{i}^{(t)} \otimes \mathsf{g}^{(t)},$$

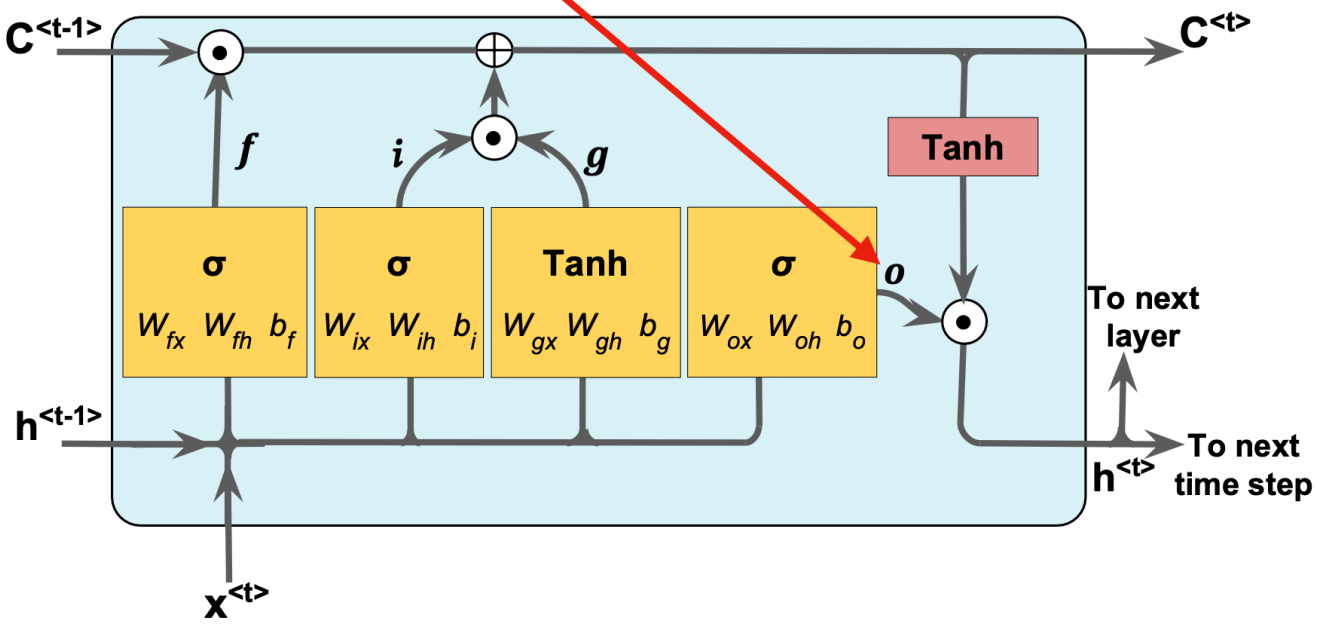
where $f^{(t)}$ and $i^{(t)}$ are the outputs of the forget gate and the input gate, respectively.

Basic layout

Long-short term memory (LSTM)

Output gate for updating the values of hidden units:

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_{ox}\mathbf{x}^{\langle t\rangle} + \mathbf{W}_{oh}\mathbf{h}^{\langle t-1\rangle} + \mathbf{b}_o\right)$$



Output gate

The final stage of the LSTM is the output gate, and its purpose is to update the short-term memory. To achieve this, we take the newly generated long-term memory and process it through a hyperbolic tangent (tanh) function creating a potential new short-term memory. We then multiply this potential memory by the output of the Sigmoid function (σ_o). This multiplication generates the final output as well as the input for the next hidden cell ($h^{\langle t \rangle}$) within the LSTM cell.
We have

$$\mathrm{o}^{(t)} = \sigma_g(W_o \mathrm{x}^{(t)} + U_o \mathrm{h}^{(t-1)} + \mathrm{b}_o),$$
$$\mathrm{h}^{(t)} = \mathrm{o}^{(t)} \otimes \sigma_h(\mathrm{c}^{(t)}).$$

where $\mathrm{W_o}, \mathrm{U_o}$ are the weights of the output gate and $\mathrm{b_o}$ is the bias of the output gate.

Summary of LSTM

LSTMs provide a basic approach for modeling long-range dependencies in sequences. If you wish to read more, see **An Empirical Exploration of Recurrent Network Architectures**, authored by Rafal Jozefowicz *et al.*, Proceedings of ICML, 2342-2350, 2015).

An important recent development are so-called noting is a more recent method named **gated recurrent unit (GRU)**, se for example the article by Junyoung Chung *et al.*,, at URL:"https://arxiv.org/abs/1412.3555. This article is an excellent read if you are interested in learning more about these modern RNN architectures

The GRUs have a simpler architecture than LSTMs. This leads to computationally more efficient methods, while their performance in some tasks, such as polyphonic music modeling, is comparable to LSTMs.

And recently Beck *et al.*, see `https://arxiv.org/abs/2405.04517`, have demonstrated that exponential gating and modified memory structures boost xLSTM capabilities to perform favorably when compared to state of the art

LSTM implementation using TensorFlow

```
"""
Key points:
1. The input images (28x28 pixels) are treated as sequences of 28 time
2. The LSTM layer processes this sequential data
3. A final dense layer with softmax activation handles the classificat
4. Typical accuracy ranges between 95-98% (lower than CNNs but reasona

Note: LSTMs are not typically used for image classification (CNNs are

To improve performance, you could:
1. Add more LSTM layers
2. Use Bidirectional LSTMs
3. Increase the number of units
4. Add dropout for regularization
5. Use learning rate scheduling
"""

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.utils import to_categorical

# Load and preprocess data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_da

# Normalize pixel values to [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

And the corresponding one with PyTorch

```
"""
Key components:
1. **Data Handling**: Uses PyTorch DataLoader with MNIST dataset
2. **LSTM Architecture**:
   - Input sequence of 28 timesteps (image rows)
   - 128 hidden units in LSTM layer
   - Fully connected layer for classification
3. **Training**:
   - Cross-entropy loss
   - Adam optimizer
   - Automatic GPU utilization if available

This implementation typically achieves **97-98% accuracy** after 10 ep
- Explicit device management (CPU/GPU)
- Manual training loop
- Different data loading pipeline
- More explicit tensor reshaping

To improve performance, you could:
1. Add dropout regularization
2. Use bidirectional LSTM
3. Implement learning rate scheduling
4. Add batch normalization
5. Increase model capacity (more layers/units)
"""

import torch
import torch.nn as nn
import torch.optim as optim
```